SIMATS ENGINEERING

DESIGN AND ANALYSIS OF ALGORITHMS

ASSESSMENT TOOL – 1

Q39. DECODE WAYS PROBLEM USING DYNAMIC PROGRAMMING

Topic: Dynamic Programming

Course Code	CSA0619
Course Name	Design and Analysis of Algorithms
Course Outcome	CO4
Programming Language	Python
Topic	Dynamic Programming
Student Name	T.Kodanda Ram
Register Number	192524162
Department	AI & DS
Academic Year	2026

Submitted in partial fulfilment of the continuous assessment requirements for the course Design and Analysis of Algorithms (CSA06).

2. PROBLEM STATEMENT, OBJECTIVE AND REQUIREMENTS

2.1 Problem Statement

Question 39 of the assessment considers a message that has been encoded from letters into digits using a fixed substitution scheme in which the letter **A** is represented by **1**, the letter **B** by **2**, and so on until the letter **Z**, which is represented by **26**. Once the letters are replaced by their numeric codes and the separators between them are removed, the resulting digit string becomes ambiguous: a given sequence of digits may correspond to several different original messages.

Given such a digit string, the task is to determine the **total number of valid ways in which the string can be decoded** back into letters. A decoding is valid only when every digit of the string is consumed by exactly one letter code, and every code used lies in the range 1 to 26. The problem explicitly requires the count to be obtained using **Dynamic Programming**, since the number of decodings of a prefix can be expressed in terms of the number of decodings of shorter prefixes.

The prescribed sample of the question is the string **226**, for which the expected output is **Ways = 3**. The implementation submitted with this report reproduces this result and, in addition, prints the three decoded strings themselves so that the correctness of the count can be visually verified by the evaluator.

2.2 Objective

The objectives of this assessment work are stated below.

No.	Objective
1	Determine the number of valid decodings of a given digit string.
2	Apply Dynamic Programming so that overlapping sub-problems are solved only once.
3	Correctly handle both one-digit (1-9) and two-digit (10-26) letter mappings.
4	Handle invalid situations such as a standalone zero and a leading zero.
5	Display the actual valid decoding combinations, not merely the count.
6	Analyse the time complexity and the space complexity of the solution.
7	Test the implementation with normal, boundary and hidden-style inputs.

2.3 Input and Output

The interface of the program is summarised in the following table.

Item	Description
Input	A digit string entered by the user (for example 226)
Valid single digit	1 - 9 (decoded as A - I)
Valid two-digit code	10 - 26 (decoded as J - Z)
Invalid standalone digit	0 (cannot represent any letter by itself)
Output	Number of valid decoding ways, followed by the decoded strings

No additional input is required from the user and no external library is used; the program depends only on the standard Python interpreter.

3. DECODING RULES AND SAMPLE EXPLANATION

The mapping between numeric codes and letters is a direct one-to-one correspondence with the position of the letter in the English alphabet. In the implementation this mapping is obtained arithmetically through the expression chr(64 + value), because the ASCII code of the character A is 65.

Number	1	2	3	4	9	10	22	25	26
Letter	A	B	C	D	I	J	V	Y	Z

Only codes from 1 to 26 exist, therefore a decoding step may consume either one digit or two digits and never more. This observation gives rise to the four rules that govern the whole algorithm.

Rule 1

A digit from 1 to 9 can always be decoded individually into one of the letters A to I.

Rule 2

Two consecutive digits can be decoded together only if the two-digit number they form lies between 10 and 26 inclusive, producing one of the letters J to Z.

Rule 3

The digit 0 cannot be decoded independently, because no letter is mapped to the code 0. A zero can only survive as the second digit of the valid pairs 10 or 20.

Rule 4

Any value greater than 26 cannot form a valid two-digit letter, so groupings such as 27, 35 or 52 must be rejected.

Explanation of the Sample 226

```
B F
F
Z
```

```
2  2  6  ->  B
22  6  ->  2  V
26 ->        B

Therefore:
Ways = 3
```

The string 226 is scanned from left to right. The first digit 2 must be taken alone as B, or combined with the second digit. If the first two digits are taken separately, the remaining digits 2 and 6 may either be split into B and F, giving **BBF**, or joined as the pair 26, which is valid and gives **BZ**. If instead the first two digits are joined as the pair 22, which is valid and equals V, the only digit left is 6, which must be taken alone as F, giving **VF**. Every branch of this reasoning has been exhausted, so exactly three decodings exist.

The full string cannot be treated as a single code, because 226 > 26 and no letter is assigned to such a value. Likewise, no grouping of three or more digits is ever considered by the algorithm for the same reason.

4. ALGORITHM USED IN THE IMPLEMENTATION

The counting algorithm used in the submitted program is the bottom-up Dynamic Programming formulation implemented inside the function decode_ways(s). Let W(i) denote the number of valid decodings of the prefix of the string consisting of its first i characters. A decoding of that prefix must end either with a single-digit letter or with a two-digit letter, and these two possibilities are mutually exclusive. Hence W(i) = W(i-1) when the last digit is non-zero, plus W(i-2) when the last two digits form a number between 10 and 26. Because only the two immediately preceding values are ever needed, the entire DP array is replaced by two scalar variables.

4.1 Variables Used

Variable	Meaning in the implementation
prev2	Number of decodings of the prefix ending two positions before the current one
prev1	Number of decodings of the prefix ending at the previous position
current	Number of decodings of the prefix ending at the current position i
two_digit	Integer value formed by the digits at positions i-1 and i

4.2 Initialisation

Before the loop begins, the implementation performs a guard check: if the string is empty or its first character is '0', the function immediately returns 0, since an encoded message can never begin with a zero. Otherwise prev2 = 1 and prev1 = 1. The value prev1 = 1 records that the single-character prefix has exactly one decoding, and prev2 = 1 represents the empty prefix, which is conventionally counted as one decoding so that a valid leading pair is counted exactly once.

4.3 The Loop

```
for i in range(1,len(s)):
```

The loop starts from index 1 because the first character has already been accounted for during initialisation. At every position the variable current is reset to 0 and the two decisions below are evaluated.

4.4 Single-digit Case

```
if s[i] != '0':
    current +=prev1
```

The digit at position i can be decoded on its own whenever it is not zero. In that case each decoding of the prefix ending at position i-1 can be extended by exactly one letter, so all prev1 decodings are carried forward.

4.5 Two-digit Case

```
two_digit = int(s[i - 1:i + 1])

if 10 <= two_digit <= 26:
    current +=prev2
```

The previous digit and the current digit are combined into a single integer. They form a valid letter only when this integer lies between 10 and 26; the lower bound automatically rejects a leading zero in the pair and the upper bound rejects values such as 27 or 52. When the pair is valid, every decoding counted in prev2 can be extended by this two-digit letter.

4.6 State Update and Result

```
prev2 = prev1
prev1 = current
...
return prev1
```

The window of two states is shifted one position to the right and the loop continues. After the final iteration `prev1` holds the number of decodings of the complete string and is returned as the answer.

5. PSEUDOCODE AND WORKING FLOW

5.1 Pseudocode

```
DecodeWays(s)

1. If s is empty or begins with 0:
       return 0

2. Set prev2 = 1

3. Set prev1 = 1

4. For every position i from 1 to n-1:
       current = 0

       If current digit is not 0:
           add prev1 to current

       Form the two-digit number
       from previous and current digits

       If two-digit number is between 10 and 26:
           add prev2 to current

       Update prev2 and prev1

5. Return prev1
```

5.2 Dynamic Programming Table for the Sample 226

The table below traces the execution of the pseudocode on the prescribed input 226. Each row shows the state after the corresponding position has been processed.

Position	Digit(s) considered	Valid single	Valid pair	prev2	prev1	Ways
Start	—	—	—	1	1	1
i = 1	2 and 2, 2	Yes (2 → B)	Yes (22 → V)	1	2	2
i = 2	6 and 2, 6	Yes (6 → F)	Yes (26 → Z)	2	3	3

5.3 How the Final Result Becomes 3

Initially both states equal 1: the empty prefix and the prefix "2" each have one decoding. At position i = 1 the digit 2 is non-zero, so the single-digit branch contributes prev1 = 1; the pair 22 lies within 10 and 26, so the two-digit branch contributes prev2 = 1. Therefore current = 2, which correctly corresponds to the two decodings BB and V of the prefix "22". The states are then shifted so that prev2 = 1 and prev1 = 2.

At position i = 2 the digit 6 is non-zero, contributing prev1 = 2, and the pair 26 is valid, contributing prev2 = 1. Consequently current = 2 + 1 = 3. The loop ends and prev1 = 3 is returned, which is precisely the expected output **Ways = 3**. The two contributions correspond exactly to the decodings obtained by appending F to BB and V, and to the decoding obtained by appending Z to B.

The essential point is that no combination is ever enumerated during counting; each position is visited once and the answer is accumulated arithmetically from the two stored states.

CODE

```python
def decode_ways(s):
    if not s or s[0] == '0':
        return 0

    prev2 = 1
    prev1 = 1

    for i in range(1, len(s)):
        current = 0

        # Single digit decoding
        if s[i] != '0':
            current += prev1

        # Two digit decoding
        two_digit = int(s[i - 1:i + 1])

        if 10 <= two_digit <= 26:
            current += prev2

        prev2 = prev1
        prev1 = current

    return prev1


def generate_decodings(s):
    result = []
```

```python
    def backtrack(index, current):
        # Complete decoding obtained
        if index == len(s):
            result.append(current)
            return

        # Single digit: 1 to 9
        if s[index] != '0':
            value = int(s[index])
            letter = chr(64 + value)
            backtrack(index + 1, current + letter)

        # Two digits: 10 to 26
        if index + 1 < len(s):
            value = int(s[index:index + 2])

            if 10 <= value <= 26:
                letter = chr(64 + value)
                backtrack(index + 2, current + letter)

    backtrack(0, "")
    return result


# Input
s = input("Enter digit string: ").strip()

# Calculate number of ways
ways = decode_ways(s)
```

```python
# Display result
print("\nNumber of Ways =", ways)

# Display all valid decoding combinations
if ways > 0:
    decodings = generate_decodings(s)

    print("\nValid Decoding Combinations:")

    for decoding in decodings:
        print(decoding)

    # Special explanation for the sample 226
    if s == "226":
        print("\nReason for 226:")
        print("2 2 6  -> B B F")
        print("22 6   -> V F")
        print("2 26   -> B Z")

        print("\nTherefore:")
        print("Ways = 3")

else:
    print("\nNo valid decoding exists.")

# Complexity
print("\nTime Complexity  : O(n)")
print("Space Complexity : O(1) for Dynamic Programming")
```

OUTPUT

```
Enter digit string: 226

Number of Ways = 3

Valid Decoding Combinations:
BBF
BZ
VF

Reason for 226:
2 2 6  -> B B F
22 6   -> V F
2 26   -> B Z

Therefore:
Ways = 3

Time Complexity  : O(n)
Space Complexity : O(1) for Dynamic Programming
```

6. PYTHON IMPLEMENTATION

6.1 Function 1 — decode_ways(s)

```
def decode_ways(s):
    if not s or s[0] == '0':
        return 0

    prev2 = 1
    prev1 = 1

    for i in range(1, len(s)):
        current = 0

        # Single digit decoding
        if s[i] != '0':
            current += prev1

        # Two digit decoding
        two_digit = int(s[i - 1:i + 1])

        if 10 <= two_digit <= 26:
            current += prev2

        prev2 = prev1
        prev1 = current

    return prev1
```

Empty input handling and leading zero: the guard if not s or s[0] == '0' returns 0 for an empty string and for any string starting with zero. **DP initialisation:** both states are set to 1. **Single-digit condition:** a non-zero digit adds prev1. **Two-digit condition:** a pair in the range 10 to 26 adds prev2. **State update:** the two variables slide forward. **Final result:** prev1 is returned. If a digit is zero and the pair it forms is invalid, current stays 0.

6.2 Function 2 — generate_decodings(s)

```
def generate_decodings(s):
    result = []

    def backtrack(index, current):
        # Complete decoding obtained
        if index == len(s):
            result.append(current)
            return

        # Single digit: 1 to 9
        if s[index] != '0':
            value = int(s[index])
            letter = chr(64 + value)
            backtrack(index + 1, current + letter)

        # Two digits: 10 to 26
        if index + 1 < len(s):
            value = int(s[index:index + 2])

            if 10 <= value <= 26:
                letter = chr(64 + value)
                backtrack(index + 2, current + letter)

    backtrack(0, "")
    return result
```

result is the list that collects the completed decoded strings. **backtrack()** is the recursive helper that builds a decoding character by character. Its **base case** is reached when index == len(s), at which point the accumulated string is a complete valid decoding and is appended to the result. The **single-digit branch** converts a non-zero digit into a letter with chr(64 + value) and recurses one position ahead; the **two-digit branch** does the same for a pair whose value lies between 10 and 26 and recurses two positions ahead. **Recursive exploration** mirrors the same two rules used by the DP function, so the length of the returned list always equals the value returned by decode_ways(s).

Important: the core counting solution uses **Dynamic Programming**, while generate_decodings() is an additional function used only to display the actual combinations. The claim of O(1) space therefore applies to the DP counting function alone, not to the storage of all generated decoding strings.

7. EXECUTION, OUTPUT AND DETAILED SAMPLE ANALYSIS

7.1 Driver Code

```
s = input("Enter digit string: ").strip()

ways = decode_ways(s)
print("\nNumber of Ways =", ways)

if ways > 0:
    decodings = generate_decodings(s)
    print("\nValid Decoding Combinations:")
    for decoding in decodings:
        print(decoding)
else:
    print("\nNovaliddecodingexists.")
```

7.2 Actual Output for the Sample Input

```
Enter digit string: 226

Number of Ways = 3

Valid Decoding Combinations:
BBF
BZ
VF

Reason for 226:
2 2 6 -> B B F -> V F
22 6 -> V F
2 26 -> B Z
26

Therefore:
Ways = 3

Time Complexity : O(n)
Space Complexity : O(1) for Dynamic Programming
```

The three combinations are listed in the order produced by the backtracking traversal, which explores the single-digit branch before the two-digit branch at every position.

7.3 Explanation of Each Result

Grouping	Codes used	Letters	Decoded string
2 2 6	2, 2, 6	B, B, F	BBF
22 6	22, 6	V, F	VF
2 26	2, 26	B, Z	BZ

In the first grouping every digit is decoded individually: 2 gives B, the second 2 gives B again and 6 gives F. In the second grouping the leading pair 22 is valid and gives V, after which the remaining digit 6 gives F. In the third grouping the first digit 2 gives B and the trailing pair 26 is valid and gives Z.

7.4 Why No Other Combination Exists

Any decoding of a three-digit string must partition it into blocks of size one or two, and only three such partitions exist: (1,1,1), (2,1) and (1,2). The partition (3) is impossible because 226 exceeds 26. All three admissible partitions were tested above and all three happened to be valid, since 22 and 26 both lie within the permitted range and no digit of the string is zero. Therefore the count of valid decodings is exactly 3, which matches the value returned by the Dynamic Programming function.

8. SECOND TEST CASE AND EDGE CASES

8.1 Test Case: 52354

```
Enter digit string: 52354

Number of Ways = 2

Valid Decoding Combinations:
EBCED
EWED

5 2 3 5 4  -> E B C E D
5 23 5 4   -> E W E D

Therefore:
Ways = 2
```

In the first grouping every digit is decoded on its own: 5 gives E, 2 gives B, 3 gives C, 5 gives E and 4 gives D, producing **EBCED**. In the second grouping the pair 23 is valid and gives W, so the decoding becomes 5, 23, 5, 4, that is **EWED**. No other grouping is possible, as shown below.

Pair	Value	Status	Reason
52	52	Invalid	Greater than 26
23	23	Valid	Lies within 10 and 26 (letter W)
35	35	Invalid	Greater than 26
54	54	Invalid	Greater than 26

8.2 Test Case Table

Input	Expected Ways	Explanation
226	3	Three valid decodings (BBF, VF, BZ)
52354	2	Two valid decodings (EBCED, EWED)
12	2	1-2 (AB) and 12 (L)
10	1	10 only, since 0 cannot stand alone
201	1	20-1 only, since 0 and 01 are invalid
27	1	2-7 only, because 27 exceeds 26
06	0	Leading zero rejected by the guard condition
100	0	Invalid zero usage: the final 0 cannot be decoded
111	3	Multiple valid groupings (AAA, KA, AK)
0	0	Zero cannot stand alone
1	1	One valid single-digit decoding (A)

8.3 How the Implementation Handles Edge Cases

A **leading zero** or an **empty string** is rejected before the loop begins, so 06 and "" return 0 immediately. An **internal zero** is never counted by the single-digit branch; it can only contribute through the two-digit branch as 10 or 20, which is why 10 and 201 return 1 while 100 returns 0, the trailing zero having no valid partner. A **pair greater than 26** is filtered by the upper bound of the range test, as seen in 27. A **single-character input** skips the loop entirely and returns the initial value 1. These behaviours cover the normal, boundary and hidden test categories expected by the Test Case Handling criterion.

9. COMPLEXITY ANALYSIS AND RUBRIC ALIGNMENT

9.1 Time Complexity

The core Dynamic Programming function decode_ways(s) executes a single loop from index 1 to n-1, where n is the length of the digit string. Inside the loop the work performed consists of one character comparison, one slice of fixed length two, one integer conversion, one range test and two assignments. Every one of these operations takes constant time and none of them depends on n. Therefore the total running time is proportional to the number of iterations and the time complexity is **O(n)**. There is no recomputation of sub-problems, which is precisely the advantage of the Dynamic Programming formulation over a plain recursive enumeration whose cost would grow exponentially.

9.2 Space Complexity

For the core DP counting algorithm the space complexity is **O(1)**, because the function stores only the three scalar variables prev2, prev1 and current, together with the temporary two_digit, instead of maintaining an entire DP array of length n + 1. This optimisation is possible because the recurrence refers only to the two immediately preceding states.

A separate and honest statement must be made about the demonstration function. generate_decodings() stores every generated decoding string in the list result, so its memory usage depends on the number of combinations and on their length, and its running time is proportional to the size of the output rather than to n alone. The claim of O(n) time and O(1) space therefore applies strictly to the counting function; the display of all combinations is an optional extra included for verification and is not part of the optimal counting solution.

Component	Time	Auxiliary space
decode_ways(s) — core DP counting	O(n)	O(1)
generate_decodings(s) — demonstration	O(output size)	O(output size)

9.3 Rubric Alignment

Assessment Criterion	Weightage	Implementation Evidence
Problem Understanding	20%	Correct decoding rules and edge-case analysis
Algorithm	25%	Dynamic Programming with optimised state storage
Code Implementation	20%	Correct and clean Python implementation
Competitive Performance	20%	O(n) DP counting and O(1) DP auxiliary space
Test Case Handling	15%	Normal and edge-case testing

The report demonstrates the Level 4 expectations of the rubric in each criterion. The constraints and edge cases are stated explicitly and justified in Sections 3 and 8. The algorithm chosen is optimal for this problem: linear time with constant auxiliary space, which cannot be improved because every digit must be examined at least once. The Python implementation is correct, readable and free of redundant structures, and it is the student's own submitted code rather than a rewritten variant. Competitive performance is evidenced by the elimination of the DP array. Finally, normal inputs, boundary inputs and hidden-style inputs involving zeros and out-of-range pairs are all tested and explained.

10. CONCLUSION AND KEY TAKEAWAYS

10.1 Conclusion

The Decode Ways problem of Question 39 was solved using Dynamic Programming, as required by the question. The submitted implementation counts the valid decoding combinations of a digit string efficiently by expressing the number of decodings of each prefix in terms of the decodings of the two shorter prefixes that precede it, so that no sub-problem is ever recomputed.

At every position the algorithm considers both possibilities allowed by the encoding scheme: the current digit may be decoded on its own when it is non-zero, and the current digit may be joined with the preceding digit when the resulting value lies between 10 and 26. Invalid two-digit values such as 52 or 35 are rejected by the range test, and zero handling prevents invalid decodings both at the start of the string and in its interior.

The prescribed sample input 226 produces exactly three ways, namely BBF, VF and BZ, matching the expected output of the question. The additional test case 52354 produces exactly two ways, namely EBCED and EWED. The optimised counting function runs in O(n) time and uses O(1) auxiliary DP space, while the supplementary function generate_decodings() reconstructs and displays the actual decoded strings so that the numeric answer can be independently verified.

10.2 Key Takeaways

1	Decode values must lie between 1 and 26; no longer grouping is ever valid.
2	Dynamic Programming avoids repeated computation of overlapping sub-problems.
3	The digit 0 requires special handling and is valid only inside 10 and 20.
4	Only two previous DP states are required, so the DP array can be discarded.
5	The core counting algorithm achieves O(n) time and O(1) auxiliary space.

10.3 Final Result

Algorithm	Dynamic Programming
Sample Input	226
Sample Output	Ways = 3
Additional Test Case	52354
Output	Ways = 2
Time Complexity	O(n)
Space Complexity	O(1) auxiliary space for the core DP counting algorithm

The assessment work is therefore complete: the problem has been understood in terms of its constraints, an optimal Dynamic Programming algorithm has been designed and justified, the Python implementation has been presented and explained function by function, its performance has been analysed, and its behaviour has been verified on normal as well as edge test cases.